GE VERNOVA

GridOS Alarm

0.7.0

Installation Guide

Copyright

Copyright 2026, GE Vernova and/or its affiliates ("GE Vernova"). All Rights Reserved.

GE and the GE Monogram are trademarks of General Electric Company used under trademark license.

This document is the confidential and proprietary information of GE Vernova and may not be reproduced, transmitted, stored, or copied in whole or in part, or used to furnish information to others, without the prior written permission of GE Vernova.

Change History

Date	Change Description
2025-Jun-18	GridOS Alarm 0.3.0: • This is a new document for GridOS Alarm 0.3.0.
2025-Jul-30	GridOS Alarm 0.4.0: • Added Exceptions. • Added Object Definitions / Exception Groups. • Added Auto-acknowledgements. • Updated Installation Overview and Installation Prerequisites with information about deployment when installing GridOS SCADA along with GridOS Alarm.

2025-Sep-16	GridOS Alarm 0.5.0:
	• Updated Alarm Model section:
	  ○ Added Categories.
	  ○ Moved processing-related information to the *GridOS Alarm User Guide*.
	• Updated Permissions to add AOR information previously in the *GridOS Alarm User Guide*.
	• Updated Alarm Service Parameters to add configuration and documentation for default values and sizing/purge parameters for alarms and events.

2026-Jan-12	GridOS Alarm 0.6.0:
	• Updated Alarm UI Setup section:
	  ○ Added Application Configuration, Menu Configuration, and Table Configuration with detailed configuration descriptions.
	  ○ Added global user configuration settings.
	• Added Alarm Test Application Parameters section.
	• Added the sympathticAutoAck attribute under Exceptions in the Alarm Model chapter.
	• Editorial and formatting updates.

2026-Mar-04	GridOS Alarm 0.7.0: • Updated Alarm Service Parameters with descriptions for alarm list sizing and purge configuration properties (`alarm.sizing. and alarm.cleanPurged.`). • Updated parameters

Contents

1. Introduction **5**
1.1. Integration with Heritage (**e-terra**) 5
1.2. About This Document 5
1.3. Purpose of This Document 6
1.4. Who Should Use This Document 6
1.5. Related Resources 6
1.6. System Overview 6
2. System Overview **7**
2.1. Key Architecture and Design Concepts 7
3. Infrastructure Prerequisites **8**
4. Installation Overview **9**
4.1. Overview of GridOS Alarm Helm Charts 9
4.1.1. What is an Umbrella Chart? 9
4.1.2. Structure of GridOS Alarm Chart 9
4.2. GridOS Alarm Services Sub-charts 9
4.3. Deploying GridOS Alarm 10
4.3.1. Prerequisites 10
4.3.2. Deployment Methods 10
4.3.2.1. Deploying GridOS Alarm Services Using Helm 10
4.3.2.1.1. Prerequisites 11
4.3.2.1.2. Command Example 11
4.3.2.2. Deploying GridOS Alarm Services Using ArgoCD 11
4.3.2.2.1. Prerequisites 11
4.3.2.2.2. ArgoCD Application YAML Structure 11
4.3.3. Global Parameters 13
4.3.3.1. Overview 13
4.3.3.2. Configuration 13
4.3.4. Alarm Service 16
4.3.4.1. Overview 16
4.3.4.2. Configuration 16
4.3.5. Alarm Router Service 18
4.3.5.1. Overview 18
4.3.5.2. Configuration 18
4.3.6. Alarm Data Service 18
4.3.6.1. Overview 18
4.3.6.2. Configuration 19
4.3.7. Alarm UI Configuration 19
4.3.7.1. Overview 19
4.3.7.2. Configuration 19
4.3.8. Alarm UI Pref-Init Configuration 20

Proprietary - See Copyright Page

4.3.8.1. Overview 20
4.3.8.2. Configuration 20
4.3.9. Alarm Schemas 20
4.3.9.1. Overview 20
4.3.9.2. Configuration 21
4.3.10. Alarm Data 21
4.3.10.1. Overview 21
4.3.10.2. Configuration 21
4.3.11. Alarm Test App 22
4.3.11.1. Overview 22
4.3.11.2. Configuration 22
5. Alarm UI Setup 23
5.1. Application Configuration 23
5.2. Global User Settings 23
5.3. Troubleshooting 24
5.4. Application Configuration 24
5.4.1. Configuration Settings 24
5.4.2. Data Source Configuration 26
5.5. Menu Configuration 27
5.6. Table Configuration 27
5.6.1. Column Configuration 27
5.6.1.1. Available Columns 28
5.6.1.2. Alarm Background Color-Coding Based on Category 30
5.6.1.3. Daylight Saving Time Indicator 30
5.6.1.4. Locking Visibility 31
6. Authorizations Overview 32
6.1. Permissions and Roles 32
6.2. Areas of Responsibility 32
7. Alarm Model 33
7.1. Exceptions 34
7.2. Object Definitions / Exception Groups 34
7.3. Categories 35
8. Account Inventory 36
8.1. Kubernetes Service Accounts 36
9. Encryption/Hashing, Tokens, and Certificates Inventory 37
10. GridOS Alarm Open and Listening Ports 38
10.1. Standard Ports 38
10.2. In-Cluster Service Ports 38

1. Introduction

GridOS Alarm is a solution that provides the following:

- Alarm Service
- Alarm Data Service
- Alarm Router Service
- Alarm UI
- Alarm UI Pref Init Package

The GridOS Alarm solution packages Helm charts and container images and delivers them through Artifactory.

Note: the term “Alarm” in this document is used interchangeably with Alarm/Event. Many of what is documented here applies to Events also.

You can install GridOS Alarm using one of two methods:

- Directly using the Helm command line.

  > **Note**
  >
  > This is a good method for prototyping or getting started with the solution. For more information, refer to Helm Commands.

- For production-grade environments, GE Vernova recommends using GitOps tools for Kubernetes.

  > **Note**
  >
  > While any reputable GitOps tool should be able to work with the Helm charts provided, GE Vernova tests with and recommends ArgoCD.

1.1. Integration with Heritage (**e-terra**)

No GridOS Alarm components should be running in the GridOS cluster when integrating to Heritage Alarm. However, a Kafka broker for receiving Alarm CloudEvents from GridOS applications must be configured on the GridOS cluster. To run in hybrid mode, configure the ALMKAFKA habitat application to consume Alarms from the GridOS Kafka.

For more information, see the *Habitat Alarm User Guide*.

1.2. About This Document

This document is supplied as a part of GE Vernova’s Kubernetes-based solution named GridOS

Proprietary - See Copyright Page

Alarm (referred to in these documents as 'Alarm' or 'ALARM')

1.3. Purpose of This Document

The purpose of this document is to provide a comprehensive guide for the installation, configuration, and maintenance of Alarm systems.

1.4. Who Should Use This Document

This document is intended for individuals who have system-level responsibilities, and also those who wish to gain a larger perspective on how the system is organized from an implementation standpoint.

1.5. Related Resources

For more information about GridOS Foundation and GridOS Data Fabric, refer to their respective documentation sets.

1.6. System Overview

GridOS Alarm is a suite of applications designed using a microservices architecture, enabling high flexibility, scalability, and adaptability to modern operational demands. These microservices are orchestrated using Kubernetes, ensuring efficient deployment, scaling, and management within a containerized environment.

The system relies on the GridOS Foundation and GridOS Data Fabric foundational layers:

- **GridOS Foundation Layer:** This layer provides essential security and monitoring capabilities. It acts as the backbone of the system, ensuring compliance, observability, and secure interactions across all services.
- **GridOS Data Fabric Layer:** Provides Anybase, a database used to store alarm modeling data.

Proprietary - See Copyright Page

2. System Overview

The purpose of the Alarm system is to provide a means for other services to send alarms and events that will be collected and presented to the operator. Alarm and event lists allow the operator to sort and filter the lists specific to their situational needs. Actions such as acknowledgement, return-to-normal, and delete are processed by the alarm service.

Alarms and Events can come from any application or service, and they are completely independent of the source definition. Once an application has generated an Alarm, it has a life of its own. The Alarm system is responsible for ensuring that no alarm or event is lost, and that the workflow defined in the alarm model for that specific alarm is enforced.

Functional permissions are applied to the Alarm system to ensure that only authorized users can view or interact with alarms and events. Areas of Responsibility (AOR) filtering is done in the Alarm Data Service to ensure that users only access alarms and events for the areas they are authorized to view and manage.

2.1. Key Architecture and Design Concepts

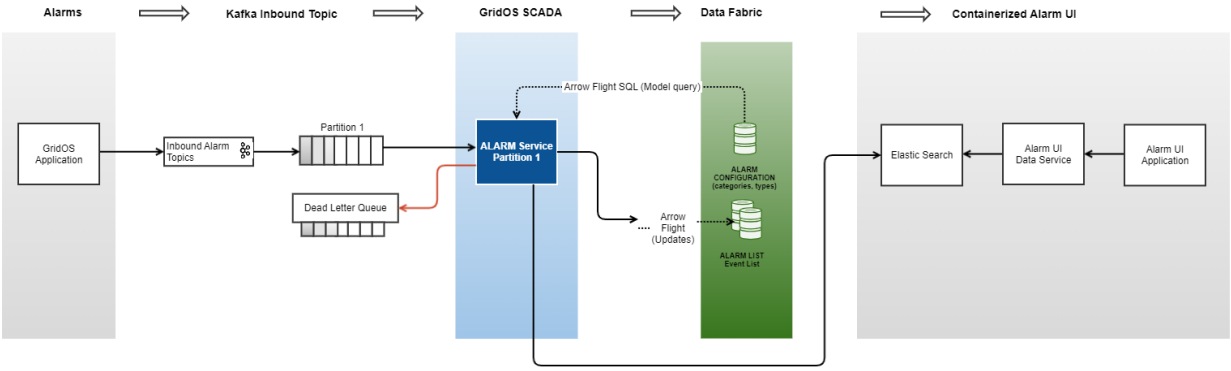


Figure 1. Architecture Overview

3. Infrastructure Prerequisites

Ensure that you prepare the following infrastructure before deploying GridOS Alarm services:

- **Kubernetes Cluster:** A fully operational Kubernetes as per Foundation documentation.
- **GridOS Foundation Layer:** Deployed and configured as the foundational base for GridOS SCADA services.

  To install GridOS Alarm, the GridOS Foundation Layer must first be deployed on the Kubernetes cluster, as it serves as a critical prerequisite. The GridOS Foundation Layer provides key components and functionalities upon which GridOS Alarm relies. It comprises multiple tiers built on a robust base layer.

  For comprehensive instructions on deploying the GridOS Foundation Layer, refer to the *GridOS Foundation Installation Guide*.
- **GridOS Data Fabric Layer:** Installed to support seamless data integration and processing.
- **Kafka Cluster:** A Kafka cluster for receiving CloudEvents from clients.

> **Important**
>
> These prerequisites are mandatory for a successful deployment of GridOS Alarm services. However, if you are deploying GridOS SCADA, please refer to the *GridOS SCADA Installation Guide* for steps to install these prerequisites and the Commons Helm chart, which includes GridOS Alarm among other services.

Proprietary - See Copyright Page

4. Installation Overview

> **Note**
>
> This topic describes the GridOS Alarm installation if you are *not* deploying Alarm along with GridOS SCADA. If you *are* deploying GridOS SCADA, instead of using the Alarm umbrella Helm chart as described in this document, GridOS Alarm is deployed using the Commons Helm chart, which also includes other services such as Kafka, Settings UI, Display Manager, UTA, and Archive Manager. For more information and applicable steps, refer to the *GridOS SCADA Installation Guide*.

4.1. Overview of GridOS Alarm Helm Charts

The GridOS Alarm deployment is managed using an umbrella Helm chart named `alarm`.

4.1.1. What is an Umbrella Chart?

An umbrella chart is a Helm chart that packages multiple related sub-charts. This structure simplifies the deployment of complex applications composed of multiple microservices, providing centralized configuration and management.

4.1.2. Structure of GridOS Alarm Chart

The `alarm` umbrella chart organizes its components as sub-charts, each representing a microservice or infrastructure component. The directory structure follows standard Helm chart conventions:

- **`Chart.yaml`:** Metadata about the chart, such as name, version, and description.
- **`values.yaml`:** Default configuration values, which can be overridden during deployment.
- **`charts/`:** Sub-charts for Alarm components.

4.2. GridOS Alarm Services Sub-charts

The `alarm` package includes the following sub-charts:

- **`alarm-service`:** Core service for processing alarms and events from Kafka.

  For more information, refer to the Alarm Service Configuration.

- **`alarm-router-service`:** Provides a REST interface for posting Alarm CloudEvents to Kafka.

  For more information, refer to the Alarm Router Service Configuration.

Proprietary - See Copyright Page

- `alarm-data-service`: Provides a GraphQL interface for querying alarm data.

  For more information, refer to the Alarm Data Service Configuration.

- `alarm-ui`: Provides a web based user interface for viewing Alarms and Events

  For more information, refer to the Alarm UI Configuration.

- `alarm-ui-pref-init`: Runs a Job to load the initial Alarm UI settings into the Preferences Service

  For more information, refer to the Alarm UI Pref Init.

- `alarm-schemas`: Runs a Job to load the schemas for Alarm into Anybase

  For more information, refer to the Alarm Schemas.

- `alarm-data`: Runs a Job to load the data for Alarm into Anybase

  For more information, refer to the Alarm Data.

- `alarm-test-generation-service`: Provides a REST interface for initiating alarm performance testing scenarios.

  For more information, refer to the Alarm Test App Configuration.

- `alarm-test-generation-ui`: Provides a web based user interface for generating Alarm performance testing scenarios.

  For more information, refer to the Alarm Test App Configuration.

4.3. Deploying GridOS Alarm

4.3.1. Prerequisites

Before deploying GridOS Alarm ensure that you have prepared the infrastructure as described in Infrastructure Prerequisites.

4.3.2. Deployment Methods

You can deploy GridOS Alarm services using:

- Helm Commands
- ArgoCD Applications

4.3.2.1. Deploying GridOS Alarm Services Using Helm

Helm is a package manager for Kubernetes that simplifies the deployment of applications.

4.3.2.1.1. Prerequisites

- Helm CLI installed.
- Kubernetes cluster configured (context set).
- Access to the Helm chart repository.

4.3.2.1.2. Command Example

To deploy the `alarm-charts` umbrella chart, enter the following command:

```
helm upgrade --install alarm alarm-charts/ \
  --namespace alarm \
  --create-namespace \
  --values alarm-values.yaml
```

Where:

- **`upgrade --install`:** Installs the chart or upgrades if it exists.
- **`release-name`:** The release name.
- **`alarm-charts/`:** Path to the GridOS Alarm Helm chart.
- **`--namespace`:** Specifies the Kubernetes namespace.
- **`--create-namespace`:** Creates the namespace if it does not exist.
- **`--set`:** Overrides specific values dynamically.
- **`--values`:** Points to the custom values file for the deployment.

4.3.2.2. Deploying GridOS Alarm Services Using ArgoCD

4.3.2.2.1. Prerequisites

Before deploying GridOS Alarm services using ArgoCD, ensure the following:

- ArgoCD is installed and operational in your Kubernetes cluster.
- The GridOS Alarm Helm chart is stored in a Git or Helm repository, accessible from ArgoCD.

4.3.2.2.2. ArgoCD Application YAML Structure

Below is a sample `Application` resource definition for deploying GridOS Alarm services:

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: alarm-services
```

```
  namespace: argocd
spec:
  project: default
  source:
    repoURL: <repository-url>
    targetRevision: <branch-or-tag>
    path: alarm-charts/
  destination:
    server: https://kubernetes.default.svc
    namespace: alarm
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

Explanation of Fields

- **Metadata:**
  - `name`: Unique name for the application in ArgoCD.
  - `namespace`: The namespace where ArgoCD is installed (default: `argocd`).
- **Source:**
  - `repoURL`: URL of the Git or Helm repository containing the GridOS Alarm chart.
  - `targetRevision`: Git branch, tag, or commit to deploy.
  - `path`: Directory path to the Helm chart or Kubernetes manifests.
- **Destination:**
  - `server`: Kubernetes API server. Use `https://kubernetes.default.svc` for in-cluster deployments.
  - `namespace`: Kubernetes namespace where GridOS Alarm services will be deployed.
- **SyncPolicy:**
  - `prune`: Automatically removes resources no longer present in the repository.
  - `selfHeal`: Automatically reconciles configuration drift.

> **Tip**
>
> Using Helm is a straightforward approach for quick and manual deployments, while ArgoCD offers robust automation, version control integration, and enhanced observability for GridOS Alarm chart deployments.

4.3.3. Global Parameters

4.3.3.1. Overview

The Global part of the umbrella chart provides the setup for common parameters used across GridOS Alarm services.

4.3.3.2. Configuration

This example base configuration should be considered as a starting point for the current stage. You should not take these values at face value; instead, use them as the starting point for final product configurations.

Global Configuration

```yaml
global:
  environment:
    name: prod
    context: realtime

  # Foundation overwrites
  clusterExternalUrl: http://ingress.local
  fqdnSuffix: .svc.cluster.local
  foundation:
    envNamespace: foundation-env-default
    monitoringNamespace: foundation-cluster-monitoring
    zeroTrustNamespace: foundation-cluster-zerotrust
  jaeger:
    enabled: true
  headerSizeLimits:
    java:
      maxHttpRequestHeaderSize: 64KB

  # Datafabric overwrites
  datafabric:
    envNamespace: data-fabric
    hellod:
      serviceName: data-fabric-hello-d
      servicePort: "8088"
    anybase:
      serviceName: anybase
      servicePort: "5000"

  # Common services overwrites
  commonServices:
    envNamespace: alarm
    environment:
      name: ""
    alarmDb: ALARMMDL
    contextApplication: COMMON
```

```yaml
# Kafka overwrites
kafka:
  commons:
    clusterName: common-cluster
    servicePort: "9092" # change port to 9093 in case enabling tls/oauth2
    topics:
      alarm:
        name: com.gridos.alarm
  maxAttempts: 10
  maxBlockMs: 6000
```

Table 1. Parameter Descriptions

Key	Type	Default	Description
`global.environment.name`	string	prod	Name of the environment SCADA services to be deployed. Kubernetes resources are distinguished based on this value.
`global.fqdnSuffix`	string	`.svc.cluster.local`	Fully Qualified DNS Name suffix. Used when referring to services deployed from different namespaces and clusters. This is provided by the GridOS Foundation layer.
`global.foundation.envNamespace`	string	`foundation-env-default`	Kubernetes namespace for Foundation general services. This is provided by the GridOS Foundation layer.
`global.foundation.monitoringNamespace`	string	`foundation-cluster-monitoring`	Kubernetes namespace for Foundation monitoring services. This is provided by the GridOS Foundation layer.
`global.foundation.zeroTrustNamespace`	string	`foundation-cluster-zerotrust`	Kubernetes namespace for Foundation zerotrust services. This is provided by the GridOS Foundation layer.
`global.jaeger.enabled`	boolean	true	Enables (true) or disables (false) jaeger tracing. This is provided by the GridOS Foundation layer.
`global.headerSizeLimits.java.maxHttpRequestHeaderSize`	string	64KB	Sets the maximum HTTP request header size. This needs to be set based on the maximum size of the bearer token issued by Foundation security services for a given deployment. This is provided by the GridOS Foundation layer.
`global.datafabric.envNamespace`	string	`data-fabric`	The Kubernetes namespace for GridOS Data Fabric services. Used when referring to GridOS Data Fabric services. This is provided by the GridOS Data Fabric layer.
`global.datafabric.hellod.serviceName`	string	`data-fabric-hello-d`	`HelloD` service name, which is used by the service to publish service discovery details. This is provided by the GridOS Data Fabric layer.

Key	Type	Default	Description
`global.datafabric.hellod.servicePort`	integer	8088	`HelloD` service port number, where HelloD listens for service discovery details. Accepted values are based on the TCP port range (0-65535). Make sure it does not conflict with ports used by other services in the system. This is provided by the GridOS Data Fabric layer.
`global.datafabric.anybase.serviceName`	string	anybase	Anybase service name, which is used by the service to persist online databases. This is provided by the GridOS Data Fabric layer.
`global.datafabric.anybase.servicePort`	integer	5000	Anybase service port number. Accepted values are based on the TCP port range (0-65535). Make sure it does not conflict with ports used by other services in the system. This is provided by the GridOS Data Fabric layer.
`global.commonServices.envNamespace`	string	alarm	Kubernetes namespace of common (shared) services, used for alarm's namespace.
`global.commonServices.environment.name`	string	""	Environment name
`global.commonServices.alarmDb`	string	ALARMMDL	Alarm database name
`global.commonServices.contextApplication`	string	COMMON	Context application name
`global.kafka.commons.clusterName`	string	`common-cluster`	Specifies the common Kafka cluster name, primarily utilized by shared services, and available for use by other services if the solution is compatible with a shared Kafka cluster. If this value is not set (left empty), then it uses the default value `foundation-cluster`.
`global.kafka.commons.servicePort`	integer	9092	Common (shared) Kafka cluster service port. Set this to `9093` in case mtls/oauth2 is enabled. Accepted values are based on the TCP port range (0-65535). Make sure it does not conflict with ports used by other services in the system.
`global.kafka.commons.topics.alarm.name`	string	`com.gridos.alarm`	Base name of the alarm Kafka topic, where the alarm-service consumes alarm messages. The effective topic name is generated by prefixing the environment name. For example, `prod.com.gridos.alarm`.
`global.kafka.maxAttempts`	integer	10	This configuration controls how many attempts can be made by a Kafka producer before raising an exception. Accepted range is `0-MAXINT`.
`global.kafka.maxBlockMs`	integer	6000	This configuration controls how long a Kafka producer will block while sending messages to a Kafka topic.

4.3.4. Alarm Service

4.3.4.1. Overview

The alarm-service section of the umbrella chart provides the configuration options for the alarm processing service.

You can find common configurations in the Global Parameters chapter.

4.3.4.2. Configuration

This example base configuration should be considered as a starting point for the current stage. You should not take these values at face value; instead, use them for the basis of final product configurations.

Alarm Service Configuration

```
alarm-service:
  enabled: true
  kafka:
    consumer:
      partition: 0
      groupid: alarm-service-groupid
  anybase:
    ddlDbName: "ALARMMDL"
  defaults:
    alarm:
      exceptionDefinition: DEFALT
      category: L2
      area: ROOT.GEOGRAPHIC.GE.MISC.NORT
  alarm:
    sizing:
      pulse:
        initDelay: 5000
        frequency: 2000
      max: 10000
      delete-percent: .2
      threshold-percent: .7
    cleanPurged:
      pulse:
        initDelay: 5000
        frequency: 90000
  event:
    sizing:
      pulse:
        initDelay: 5000
        frequency: 2000
      max: 10000
      delete-percent: .2
      threshold-percent: .7
```

Table 2. Parameter Descriptions

Key	Type	Default	Description
enabled	boolean	true	If set to `true`, the `alarm-service` shall be deployed.
replicaCount	integer	1	The number of pods that will run the `alarm-service` in the cluster.
kafka.consumer.partition	integer	0	`alarm-service` Kafka consumer partition.
kafka.consumer.groupid	string	"alarm-service-groupid"	`alarm-service` Kafka consumer groupid.
defaults.alarm.exceptionDefinition	string	DEFALT	Default exception definition used for alarms if not specified elsewhere or invalid exception definition is provided.
defaults.alarm.category	string	L2	Default category used for alarms if not specified elsewhere or invalid category is provided.
defaults.alarm.area	string	ROOT.GEOGRAPHIC.GE.MISC.NORT	Default AOR assigned to alarms for filtering purposes.
alarm.sizing.pulse.initDelay	integer	5000	Initial delay (ms) before the alarm purge routine first runs after service startup.
alarm.sizing.pulse.frequency	integer	2000	How often (ms) the alarm purge routine runs to check whether the alarm list has reached the threshold and, if so, mark alarms for removal.
alarm.sizing.max	integer	10000	Maximum number of alarms allowed in the alarm list.
alarm.sizing.delete-percent	float	0.2	Fraction of `alarm.sizing.max` to remove per purge cycle (e.g. `0.2` removes 20% of the maximum). Acknowledged alarms are purged first; unacknowledged alarms fill any remaining quota and are force-acknowledged before removal.
alarm.sizing.threshold-percent	float	0.7	Fraction of `alarm.sizing.max` at which the purge routine triggers (e.g. `0.7` triggers at 70% capacity). When the number of alarms meets or exceeds this value, a purge cycle begins.
alarm.cleanPurged.pulse.initDelay	integer	5000	Initial delay (ms) before the purged-alarm delete routine first runs after service startup.
alarm.cleanPurged.pulse.frequency	integer	90000	How often (ms) the purged-alarm delete routine runs to physically remove alarms that have been marked for purging. This runs independently of the purge routine to allow brief retention of purged alarms before permanent deletion.
event.sizing.pulse.initDelay	integer	5000	Initial delay (ms) before starting event sizing pulse operations.
event.sizing.pulse.frequency	integer	2000	Frequency (ms) of event sizing pulse operations.
event.sizing.max	integer	10000	Maximum number of events allowed in the system.
event.sizing.delete-percent	float	0.2	Percentage of events to delete when the maximum threshold is reached.
event.sizing.threshold-percent	float	0.7	Percentage threshold to trigger event management actions.

> **Note**
>
> For details on how the alarm purge and delete routines behave at runtime — including alarm selection priority, force-acknowledgement of unacknowledged alarms, and the system event generated on each purge — see the "Alarm List Size Management" section in the *GridOS Alarm User Guide*.

4.3.5. Alarm Router Service

4.3.5.1. Overview

The alarm-router-service section of the umbrella chart provides the configuration for the alarm router service.

You can find common configurations in the Global Parameters chapter.

In particular, the global parameters for the kafka topic are used by the router service.

4.3.5.2. Configuration

This example base configuration should be considered as a starting point for the current stage. You should not take these values at face value; instead, use them as the starting point for final product configurations.

Alarm Router Service Configuration

```
alarm-router-service:
  enabled: true
```

Table 3. Parameter Descriptions

Key	Type	Default	Description
alarm-router-service.enabled	boolean	true	If set to `true`, `alarm-router-service` shall be deployed.

4.3.6. Alarm Data Service

4.3.6.1. Overview

The alarm-data-service section of the umbrella chart provides the configuration for the alarm data service.

You can find common configurations in the Global Parameters chapter.

4.3.6.2. Configuration

This example base configuration should be considered as a starting point for the current stage. You should not take these values at face value; instead, use them as the starting point for final product configurations.

Alarm Data Service Configuration

```
alarm-data-service:
  enabled: true
  aors:
    enabled: true
    version: 3
  permissions:
    enabled: true
```

Table 4. Parameter Descriptions

Key	Type	Default	Description
alarm-data-service.enabled	boolean	true	If set to `true`, `alarm-data-service` shall be deployed.
alarm-data-service.aors.enabled	boolean	true	If set to `true`, `alarm-data-service` shall check AORs during queries.
alarm-data-service.aors.version	integer	3	The version of the AOR Manager api to use. Only 3 is currently supported.
alarm-data-service.permissions.enabled	boolean	true	If set to `true`, `alarm-data-service` shall check for alarms and events read permission individually.

4.3.7. Alarm UI Configuration

4.3.7.1. Overview

The alarm-ui section of the umbrella chart provides the setup for the Alarm UI configuration.

You can find common configurations in the Global Parameters chapter.

4.3.7.2. Configuration

This example base configuration should be considered as a starting point for the current stage. You should not take these values at face value; instead, use them as the starting point for final product configurations.

Alarm UI Configuration

```
alarm-ui:
  enabled: true
```

Table 5. Parameter Descriptions

Key	Type	Default	Description
alarm-ui.enabled	boolean	true	If set to `true`, `alarm-ui` shall be deployed.

4.3.8. Alarm UI Pref-Init Configuration

4.3.8.1. Overview

The alarm-ui-pref-init section of the umbrella chart provides the setup for initial Alarm UI configuration to be deployed to the Foundation Preferences Service.

You can find common configurations in the Global Parameters chapter.

4.3.8.2. Configuration

This example base configuration should be considered as a starting point but should be customized for your deployment.

Alarm UI Pref-Init Configuration

```
alarm-ui-pref-init:
  enabled: true
  preferencesService:
    cfgEdits: '.usePriorityColors = true | .productKey = \"gridos\"'
```

Table 6. Parameter Descriptions

Key	Type	Default	Description
alarm-ui-pref-init. enabled	boolean	true	If set to `true`, `alarm-ui-pref-init` shall be deployed, preferences configuration will be overwritten. This should only be enabled for initial setup.
alarm-ui-pref-init. preferencesService. cfgEdits	boolean	string	The configuration edits to be applied to the Preferences Service. This is a jq-style edit string. See the jq Manual for more information.

4.3.9. Alarm Schemas

4.3.9.1. Overview

The alarm-schemas section of the umbrella chart provides the configuration options for loading the alarm schemas.

You can find common configurations in the Global Parameters chapter.

4.3.9.2. Configuration

Alarm Schemas Configuration

```
alarm-schemas:
  enabled: true
  anybase:
    ddlDbName: "ALARMMDL"
```

Table 7. Parameter Descriptions

Key	Type	Default	Description
alarm-schemas.enabled	boolean	true	If set to `true`, the `alarm-schemas` shall be deployed.
alarm-schemas. anybase.ddlDbName	string	"ALARMMDL"	`alarm` anybase DB schema name.

4.3.10. Alarm Data

4.3.10.1. Overview

The alarm-data section of the umbrella chart provides the configuration options for loading the alarm model.

You can find common configurations in the Global Parameters chapter.

4.3.10.2. Configuration

This chart runs a job that loads the alarm model into the Anybase database. This is a one-time operation that should be performed before deploying the alarm-service. It can be redeployed to reset the model, or to load a new model if needed.

Alarm Data Configuration

```
alarm-data:
  enabled: true
```

Table 8. Parameter Descriptions

Key	Type	Default	Description
alarm-data.enabled	boolean	true	If set to `true`, the `alarm-data` shall be deployed.

4.3.11. Alarm Test App

4.3.11.1. Overview

The alarm-test section of the umbrella chart provides the configuration for the Alarm Test Application and related service.

For information about this application, refer to the *Alarm Test Application User Guide*.

> **Important**
>
> This is a test tool for use by GE Vernova for testing and validation purposes, including:
>
> - Alarm functionality
> - Configuration settings
> - Performance
>
> It is not a licensed application for customer use.
>
> This tool is **designed for use in pre-production environments only**, and should be disabled in production environments.

You can find common configurations in the Global Parameters chapter.

In particular, the global parameters for the kafka topic, alarm anybase database, and the AOR Manager service are used by the alarm-test generation service.

4.3.11.2. Configuration

To enable or disable the alarm-test generation service and UI, set the `enabled` key to `true` or `false` respectively. The default deployment configuration is shown below.

Alarm Test App Configuration

```yaml
alarm-test-generation-service:
  enabled: false

alarm-test-generation-ui:
  enabled: false
```

Table 9. Parameter Descriptions

Key	Type	Default	Description
alarm-test-generation-service.enabled	boolean	false	If set to `true`, `alarm-test-generation-service` shall be deployed.
alarm-test-generation-ui.enabled	boolean	false	If set to `true`, `alarm-test-generation-ui` shall be deployed.

5. Alarm UI Setup

This chapter describes specific customization options.

5.1. Application Configuration

The `Alarm-UI-Pref-Init` container loads a set of JSON files into the Preferences Service to configure the Alarm UI client. You can modify the settings at deployment time or after deployment using the `settings-ui`."

The main configuration file, `alarm-ui-app-config.json`, contains various application settings defining the core appearance and behavior of the application, including data source configuration.

There is also a table configuration and a menu configuration for each view (alarms, events), as well as configuration files for the priority and state buttons.

For more information about application level configuration, including datasources and core functionality, see Application Configuration.

For more information about table and menu configuration, see Table Configuration and Menu Configuration.

5.2. Global User Settings

You can configure some settings across applications using the `/user-settings/` preference. These settings include time format, date format, and time zone.

If you do not provide these settings, the system will apply the default fallback settings as outlined below:

- **timezone:** Specify the time zone using IANA identifiers, (for example, `America/New_York`, `Europe/London`, `Asia/Tokyo`).

  If not specified, the system uses the browser's local time zone by default.

- **dateTimeFormat:** Defines the date and time format using a valid `moment.js` format string.

  If not specified, the system uses the date and time format from the Alarm UI application configuration file.

- **statusBarDateTimeFormat:** Defines the date and time format used in the status bar. It accepts either predefined keywords or `moment.js` format strings. Supported keywords include: `short`, `regular`, and `long`.

  - **Short format (`short`/`short24h`):** Displays hours and minutes, followed by AM/PM if applicable.

    For example: 2:30 PM.

- **Regular format (`regular`/`regular24h`):** Displays hours, minutes, and seconds, followed by AM/PM if applicable.

  For example: 2:30:15 PM EST.

  > **Note**
  >
  > If no format is specified, the 24-hour format is used be default.
  >
  > For example, 14:30:15 EST.

- **Long format (`long`/`long24h`):** Displays the day, hours, minutes, and seconds, followed by AM/PM if applicable.

  For example: Tuesday, 2:30:15 PM EST.

5.3. Troubleshooting

Issue: The Alarm UI client is not connected to the server, or another issue has occurred.

To diagnose and resolve this issue, follow these steps:

1. **Open the Browser Console:** Use the **Developer Tools** option from your browser menu to open the console log. This log is useful for troubleshooting any errors encountered by the client.
2. **View Log Messages:** Switch to the **Console** tab to view the log messages. These messages can provide insights into what might be causing the connection issue or other problems.

5.4. Application Configuration

The main configuration contains various application settings defining the core appearance and behavior of the alarm UI, including the data source configuration.

5.4.1. Configuration Settings

The following table lists the configuration variables in the `app-config` file and their purposes.

Setting Name	Purpose
alarmActionChunkSize	The maximum number of alarms that are sent to the alarm subsystem in a single request. The default is `5`. This means if you attempt to acknowledge 25 alarms, the system will send 5 requests to the alarm subsystem, each containing 5 alarm MRIDs.

Setting Name	Purpose
alarmDocumentBaseUrl	Specifies the base URL for the alarm document server, which is where alarm operator guides are typically stored.
alarmQueueCautionPercent	The percent value at which the alarm queue status indicator becomes yellow.
alarmQueueWarningPercent	The percent value at which the alarm queue status indicator becomes red.
autoSizeColumns	If set to `true`, the table columns will auto-size when the table is loaded.
commandOperation ReportingTimeoutMs	The timeout value for command operations (in milliseconds).
dataSources	Section to define the sources of data the Alarm UI can connect to. For more information, see Data Source Configuration.
dateTimeFormat	The date/time format for the time-related columns in the Alarm UI grid.
enableAuthorizations	If set to `true`, the Alarm UI checks the user's AORs and permissions for read and write operations.
enableUrlAppState	If set to `true`, the state of the application is saved in the URL. If set to `false`, the state is not present in the URL.
localeId	Set the locale of the language displayed in the Alarm UI (for example, `en-US`, `fr`). The default value is `en-US`.
logVerbosity	Logger Verbosity Level. Options in order of verbosity: • Normal • Verbose • debug
maxAlarmQueryCount	Specifies the maximum number of rows a query result can have. The default value is `10K` if not specified, or if the specified value is less than 1000. **Note** There might be a delay in updating the Alarm UI if this value is lower than the rate at which the alarm is updated on the server, multiplied by the Alarm UI refresh rate.
maxAlarmsPerTransaction	The maximum number of alarms that can operate at a time. Alarm UI has features for grouping related alarms that make it easy to operate a group of alarms with a single click. However, for instance, if a group of alarms has 2,000 children and an operator clicks **ack**, the underlying alarm subsystem would be overwhelmed with the work, and the client would detect multiple timeouts. To avoid disruptions and ensure mistakes are minimized, this value can be set to a reasonable number, such as 10 or 20. The default value is `50`.

Setting Name	Purpose
maxConnectionRetryCount	Since the UI is based on a connectionless HTTP protocol, the notion of "connectivity" is supported by the alarm subsystem writing a heartbeat timestamp to the intermediary data cache. The alarm client polls continually for this heartbeat change. When it continues to be updated, the client is considered "connected." If a heartbeat is missed, the connection is declared as suspect, and retry logic begins. This value determines how many times the system retries (how many consecutive polls can detect a bad heartbeat timestamp) before the client declares the connection as failed and issues a warning message to the operator. The default value is `5`.
notificationMessageDuration	The duration (in milliseconds) for which notification messages are displayed in the Alarm UI.
notifyOnMessageTypes	A list of message types that will trigger a notification in the Alarm UI.
priorityColors	Specifies the color of each priority. This is used to color code the priority filter buttons (when enabled).
pauseWatermarkConfig	Configuration for the pause watermark.
showDstRepeatedHourIndicator	If set to `true`, an asterisk appears next to timestamps that fall within the repeated hour caused by the end of daylight saving time. You can customize the asterisk by modifying the `dstRepeatedHourIndicator` property for the `Time` column in the table configuration file.
usePriorityColors	If set to `true`, priority colors are used to color code priority filter buttons and the category column color bar. If set to `false`, the alarm category color configured in the Alarm application settings is used. In this case, the priority filter buttons are not color coded.

5.4.2. Data Source Configuration

This section describes the Alarm UI data source configuration. The `app-config` file can contain one or more data source elements that are possible sources of alarm data for the Alarm UI.

Each data source has the following properties:

Property	Purpose
id	Unique identifier.
description	Friendly name shown in the UI.
commandUrl	The URL path of the command service (if available). It is appended to the baseUrl to form the full URL.
usePolling	Indicates whether the Alarm UI polls the datasource at a specific interval.
pollingInterval	The polling interval in milliseconds.
port	The port number of the datasource (optional).
path	The URL path to the datasource (appended to the base path).
enabled	Indicates whether the datasource (and corresponding view) is enabled or disabled.

5.5. Menu Configuration

Each table configuration file has an associated menu configuration file that defines the context menu that appears when you right-click on an alarm in the table.

Example menu item for acknowledging an alarm:

The menu item is disabled unless the `canAck` condition is set to `true`.

The `ack_alarm` action is executed when the menu item is selected.

```
{
  "label": "Acknowledge",
  "enabled": false,
  "conditionalValue": {
    "condition": "canAck",
    "value": {
      "enabled": true
    }
    },
    "action": {
      "type": "ALARM",
      "id": "ack_alarm"
    }
}
```

5.6. Table Configuration

This chapter describes the table configuration. Each view of the Alarm UI has its own table configuration and corresponding menu configuration. This allows users to customize which columns to display and which actions are available in each view.

5.6.1. Column Configuration

Each table configuration file has a section named `columns` that contains the column definitions for the table. Columns can be as simple as displaying the text of an alarm field, or as complex as showing icons based on specific conditions and trigger actions when these icons are clicked.

The table configuration files include examples of each type of column configuration. Rather than documenting all the column definition variations, refer to the table configuration files for examples.

Some columns use a named condition to control the style of a column. The `namedConditions` section contains all the conditions for the table and the `gridOptions` section contains fields for controlling the behavior and appearance of the table.

Proprietary - See Copyright Page

5.6.1.1. Available Columns

The following table lists the available columns in the Alarm UI, along with a brief description. The field name column corresponds to the field name in the alarm/event data.

Column Name	Field Name	Description	Shown By Default?
Selection Column	N/A	A column with checkboxes to select rows, including a header checkbox to select all rows.	✓
Related Column	N/A	When grouping alarms by parent/child relationship, this column displays the number of alarms and unacknowledged alarms (in parentheses) in the group.	✓
Acknowledge/Delete Button Column	N/A	Contains a button for acknowledging or deleting the alarm.	✓
Open SLD	N/A	An icon column to open the Single Line Diagram (SLD) for the associated object.	✓
Open Tabular	N/A	An icon column to open a tabular view for the associated object.	✓
Annunciation Time	annunciationTime	Displays the time of annunciation in a formatted date-time string.	✗
Annunciation Time Nano	annunciationTimeNano	Stores the nanoseconds part of the annunciation time.	✗
Annunciation Count	annunciationCount	Displays the count of annunciations.	✗
Time	time	Displays the time of the alarm in a formatted date-time string. Includes conditional styling for abnormal alarms. By default, the time text is shown in red for abnormal alarms.	✓
Time Nanoseconds	timeNanosec	Stores the nanoseconds part of the time field.	✗
Audible	audible	An icon column indicating whether the alarm is audible. A volume icon appears when `true`.	✓
Status	state	Displays the status of the alarm.	✓
Note	noteAttached	An icon column for viewing or editing notes attached to the alarm. Visibility is conditional based on the presence of a note.	✓
Category	category	Displays the category of the alarm.	✓
Message	text	Displays the alarm message.	✓
Priority	priority	Displays the priority of the alarm.	✓
Location	location	Displays the location of the alarm.	✓
MRID	mrid	Stores the unique identifier of the alarm.	✗
Parent MRID	parentMRID	Stores the unique identifier of the parent alarm.	✗
ObjectDef	objectDef	Stores the object definition type (for example, POINT and ANALOG).	✗
Child	isChildAlarm	Indicates whether the alarm is a child alarm.	✗

Column Name	Field Name	Description	Shown By Default?
Time Acknowledged	ackTime	Shows the time when the alarm was acknowledged in a formatted date-time string.	✗
Color	color	Stores the color associated with the alarm.	✗
Category Color	categoryColor	Stores the background color of the category.	✗
Category Foreground Color	categoryForegroundColor	Stores the foreground color of the category.	✗
Composite Key	compositeKey	Stores the composite key for the alarm.	✗
Composite ID	compositeID	Stores the composite ID for the alarm.	✗
Domain	domain	Stores the domain of the alarm.	✗
Field Time	fieldTime	Displays the field time of the alarm in a formatted date-time string.	✗
Field Time Nanoseconds	fieldNanosec	Stores the nanoseconds part of the field time.	✗
Logs	logs	Stores logs associated with the alarm.	✗
Permission Areas	permissionAreasShort	Shows the permission areas associated with the alarm as a comma-separated string. By default, the short area names are displayed. To display the full name, change the field name to `permissionAreas`. For more information, see Areas of Responsibility.	✓
Unack Areas	unackAreas	Displays the unacknowledged areas associated with the alarm as a comma-separated string.	✗
Unpurged Areas	unpurgedAreas	Displays the unpurged areas associated with the alarm as a comma-separated string.	✗
Relative Time	relativeTime	Displays the relative time of the alarm using a custom renderer.	✗
Sequence Number	sequenceNum	Stores the sequence number of the alarm.	✗
Source	source	Stores the source of the alarm.	✗
Station	stationID	Stores the station ID associated with the alarm.	✗
User	userID	Stores the user ID associated with the alarm.	✗
Abnormal	abnormal	Indicates whether the alarm is abnormal.	✗
Alarm MRID	alarmMRID	Stores the unique identifier for the alarm instance.	✗
Client State	clientState	Stores the client state of the alarm.	✗
Exception Definition	exceptionDef	Stores the exception definition for the alarm.	✗
Non-deletable	nonDeletable	Indicates whether the alarm is non-deletable.	✗
Object MRID	objectMRID	Stores the unique identifier for the object associated with the alarm.	✗

Column Name	Field Name	Description	Shown By Default?
PC Name	pcName	Stores the PC name from which users perform an action on an alarm, such as acknowledging or deleting it.	✕
Severity	severity	Stores the severity level of the alarm.	✕
URL	url	Stores the URL associated with the alarm. This is used for opening the operator guide associated with the alarm.	✕

5.6.1.2. Alarm Background Color-Coding Based on Category

You can color-code the background of columns in the alarm table using the alarm category color. To do this, set the following property in the definition of each column you want to color-code.

`"conditionalValue": "categoryColorFormat"`

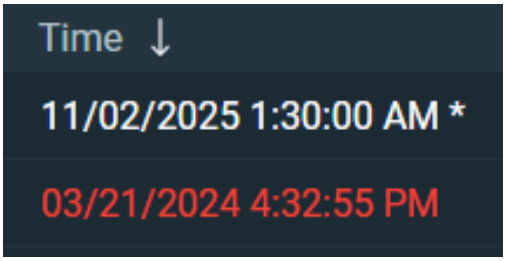
The `alarms-table` that is included in the GridOS Alarm installation contains a sample configuration. Search for the term `categoryColorFormat` and its usages to understand how it is configured.

> **Note**
>
> A column configured to use a cell renderer cannot have additional formatting applied. Therefore, the background of such columns cannot be color-coded.

```
{
  "headerName": "Message",
  "field": "text",
  "minWidth": 400,
  "maxWidth": 500,
  "flex": 1,
  "autoHeight": true,
  "lockVisible": true,
  "cellStyle": {
    "color": "var(--dex-color-text-interaction)",
    "white-space": "pre",
    "font-family": "monospace"
  },
  "_conditionalValue": "categoryColorFormat"
}
```

5.6.1.3. Daylight Saving Time Indicator

You can change the Daylight Saving Time (DST) indicator, which appears in the **Time** column for alarms created during the repeated hour, by using the `dstRepeatedHourIndicator` property in the **Time** column definition.




```
{
  "headerName": "Time",
  "field": "time",
  "type": "dateTime",
  "readonly": true,
  "format": "MM/DD/YY HH:mm:ss",
  "sort": "desc",
  "lockVisible": true,
  "conditionalValue": {
    "condition": {
      "property": "abnormal",
      "operator": "eq",
      "value": "true",
      "modifier": "boolean"
    },
    "value": {
      "style": {
        "color": "var(--dex-color-text-destructive)"
      }
    }
  }
}
```

5.6.1.4. Locking Visibility

You can configure columns to prevent users from hiding them. To do so, select the `lockVisible` property in the column definition within the table configuration.

6. Authorizations Overview

The Alarm Services and the Alarm UI support two forms of authorization: permissions and areas of responsibility (AoR). These two forms of authorization work together to control what actions a user can perform on an alarm.

6.1. Permissions and Roles

Permissions control individual actions that users can perform. Roles define a set of permissions granted to a User Group. The roles and permissions are defined with the Foundation Role Manager application.

There are several permissions that are expected for an operator in order to use Alarm and the Alarm UI. The following table describes each permission and what actions you can perform when permission is assigned.

Action	Name	Description
View alarms	alarm.read.alarms	Access to view alarms
View events	alarm.read.events	Access to view events
Acknowledge alarms	alarm.write.ackAlarm	Access to acknowledge alarms
Delete alarms	alarm.write.deleteAlarm	Access to delete alarms

6.2. Areas of Responsibility

An Area of Responsibility (AoR) represents a portion of operational responsibilities that has been assigned to a user. Those responsibilities are normally defined by geographic area, device type, network level, or a combination of the above.

Each Alarm is associated with one or more AoRs, which are defined in the `permissionAreas` field of the alarm. A user must have the appropriate permissions for at least one of the AoRs associated with an alarm in order to view (`read`) or take action (`write`) on that alarm.

The AOR data is provided by the AoR Manager service, which is configured as part of the GridOS system deployment. For more information, see the *Foundation Base User Guide*.

7. Alarm Model

The Alarm Process consumes alarm events from Kafka and joins them with the Alarm Model data stored in Anybase. The Alarm Model is split into:

- Object
- Exception
- Category

Object and Exception configuration determines how an alarm is processed, while the Category defines the presentation of an alarm.

The details of the Alarm Model are described below.

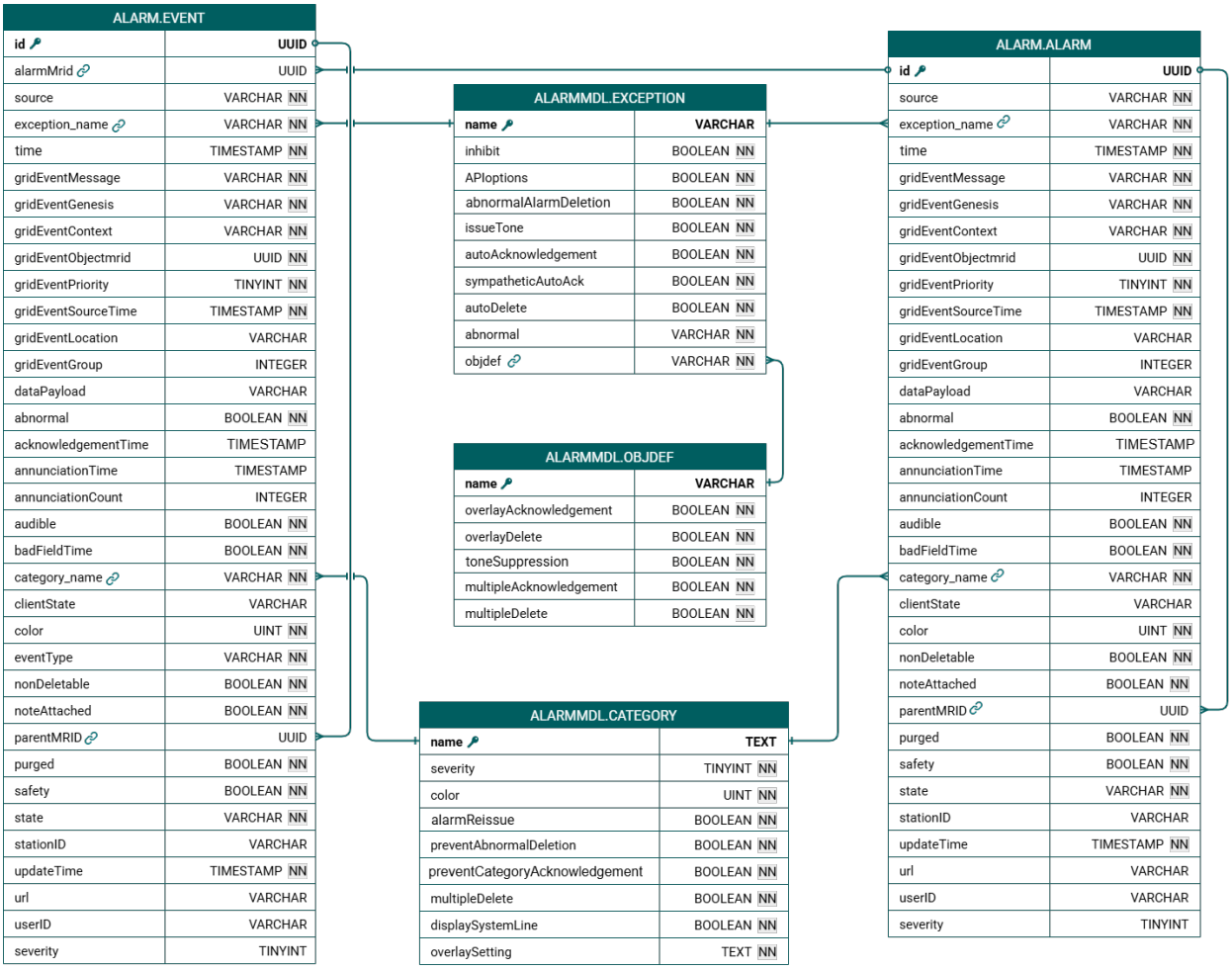


Figure 2. Schema Diagram

For more information about the Alarm Process, refer to the "Alarm Processing" chapter in the *GridOS Alarm User Guide*.

7.1. Exceptions

An exception in the alarm model defines how a specific type of alarm is processed by the system. Exceptions are linked to object definitions, allowing multiple exceptions to be grouped and managed together.

Attribute	Type	Description
name	TEXT	A string used to identify the exception.
inhibit	Boolean	**Inhibit:** When selected, indicates that the alarm for this exception is suppressed.
useApiOptions	Boolean	**Use the API options:** Indicates whether the options specified by the client, when it posts the event, should be used to process the event.
notAnAlarm	Boolean	**Not in the list:** If the API options are not used, this flag is used to decide if the event is an alarm or not.
autoAcknowledgement	Boolean	**Auto-ack:** If set, the incoming alarm is acknowledged automatically, as well as the related alarms if "multiple ack" is set on the object definition.
sympathticAutoAck	Boolean	**Sympathetic auto-ack:** If set, the incoming alarm is acknowledged automatically in the same AORs as the most recent related alarm.
autoDelete	Boolean	**Auto-delete:** If set, when the alarm is acknowledged it is also deleted, as well as related alarms if "multiple delete" is set on the object definition.
abnormalAlarmDeletion	Boolean	**Allow abnormal alarm deletion:** If set, an incoming alarm or event that is not abnormal causes deletion restrictions on related alarms to be removed. These related alarms are still abnormal, but the operator can delete them.
abnormal	Enum (YES, NO, Conditional)	**Abnormal:** Indicates whether the event is abnormal. • **YES:** Always abnormal • **NO:** Always normal • **Conditional:** The client determines the state
issueTone	Boolean	**Issue tone:** If not specified, the Alarm server does not issue a tone for the alarm.
objdef	TEXT	**Object Definition:** Links the exception to its parent object definition. Multiple exceptions can be associated with a single object definition, allowing for grouped alarm processing and configuration.

7.2. Object Definitions / Exception Groups

The object definition (`objdef`) record consists of a set of attributes that describe an Alarm database object. These attributes are used by the Alarm server when processing events associated with the object.

Attribute	Type	Description
name	TEXT	A string used to identify the object definition.

Attribute	Type	Description
multipleAcknowledgement	Boolean	**Multiple acknowledgment:** If defined, acknowledgment of any one of the alarms for the same object definition causes all related alarms to be acknowledged.
multipleDelete	Boolean	**Multiple delete:** If defined, deletion of an alarm causes all alarms for the same object definition to be deleted as well.
overlayAcknowledgement	Boolean	**Overlay/Ack:** If defined, only one unacknowledged alarm for the same object definition is allowed. A new unacknowledged alarm automatically acknowledges the previous one.
overlayDelete	Boolean	**Overlay/Del:** If defined, only one alarm (acknowledged or unacknowledged) for the same object definition is allowed. A new unacknowledged alarm automatically acknowledges and deletes the previous one.
toneSuppression	Boolean	**Multiple tone suppression:** If set, the Alarm server does not issue a second tone if a tone has already been issued for the same object definition.

7.3. Categories

Attribute	Type	Description
severity	Integer	
color	Integer	This color value is used to display the alarm in the Alarm List.
abnormalDeletion	Boolean	**Prevent Abnormal Alarm Deletion:** If this option is set on the Category record for an abnormal alarm, the alarm cannot be deleted. This should be used to prevent accidental deletion of alarms that are considered serious.

8. Account Inventory

Alarm requires no interactive user accounts in order to function. However, there are a number of credentials which are used for identification and security within the system.

8.1. Kubernetes Service Accounts

See https://kubernetes.io/docs/tasks/configure-pod-container/configure-service-account/

A Kubernetes service account is used to identify communication between pods inside of Kubernetes. They are not (and can not be) used by users. They are represented by an account 'token' which is generated at the time of creation. The Service Account does not itself allow/prevent access to resources. It is used in conjunction with Role Based Access Control (RBAC) or Istio Service Mesh AuthorizationPolicy to assign what a Kubernetes Pod can/can’t do.

At any given time, you can query the ServiceAccounts registered in Kubernetes cluster using:

```
kubectl get sa --all-namespaces
```

9. Encryption/Hashing, Tokens, and Certificates Inventory

Table 10. Tokens Inventory

Token Name	Purpose	Persistence	Protection	Source	Comments
alarm-service	Service account from GE Vernova	Until the component is redeployed	AESCBC	etcd	Verify using kubectl get serviceaccount --all -namespaces
alarm-router-service	Service account from GE Vernova	Until the component is redeployed	AESCBC	etcd	Verify using kubectl get serviceaccount --all -namespaces
alarm-data-service	Service account from GE Vernova	Until the component is redeployed	AESCBC	etcd	Verify using kubectl get serviceaccount --all -namespaces
alarm-ui	Service account from GE Vernova	Until the component is redeployed	AESCBC	etcd	Verify using kubectl get serviceaccount --all -namespaces
alarm-ui-pref-init	Service account from GE Vernova to load preferences	Until the component is redeployed	AESCBC	etcd	Verify using kubectl get serviceaccount --all -namespaces

10. GridOS Alarm Open and Listening Ports

Depending on the applications deployed into the Kubernetes Cluster, there could be a different set of ports exposed from the K8s Cluster. Many of these ports can be configured and overridden when required, so the actual results on a given deployment may differ.

10.1. Standard Ports

Standard node ports are described in the *GridOS Foundation Installation Guide*.

10.2. In-Cluster Service Ports

The following is a list of all `Service` ports open in a standard "all" deployment of GridOS Alarm. Depending on deployment configuration, not all the listed Services will be deployed/available.

On a standard RKE2 deployment, these ports are *not* available to callers from outside the cluster. Inside the cluster, they may be *reachable*, but are all protected by an Istio Sidecar Proxy that further determines whether communication is allowed to reach the business service container in the Pod (either by JWT, namespace, or service account).

Namespace	Service Name	Port	Port Name	Port Protocol
alarm	alarm-service	80	http	TCP
alarm	alarm-service	9000	http-metrics	TCP
alarm	alarm-data-service	80	http	TCP
alarm	alarm-data-service	11110	http-metrics	TCP
alarm	alarm-router-service	80	http	TCP
alarm	alarm-router-service	9001	http-metrics	TCP
alarm	alarm-ui	80	http	TCP